Escola Tècnica Superior d'Enginyeria de Telecomunicació de Barcelona
UNIVERSITAT POLITÈCNICA DE CATALUNYA

Funcions

Programació i Estructures de Dades (PRD)

Grau en Enginyeria Electrònica de Telecomunicació

ETSETB Telecom BCN

Programació modular

- Quan la **complexitat i mida del programa augmenta**, és apropiat **dividir** aquest **en subprogrames** més senzills i tractables
  - Aquesta manera de procedir rep el nom de **programació modular**
  - Sent **cada mòdul** (subprograma) independent, **podem codificar-lo sense necessitar una visió general del programa complet** ☺
  - A més, ens facilitarà la **reutilització de codi**, doncs **un mateix mòdul** (subprograma) **podrà ser cridat des de múltiples parts del programa complet** quan es requereixi ☺
- En **llenguatge de programació C**, aquests **mòduls s'anomenen funcions**, que **reben unes dades d'entrada** per realitzar la tasca encarregada i **retornen el resultat** d'aquesta

Definició i crida a funció (1/3)

- Una funció és un conjunt de sentències, al qual se li assigna un nom, que realitza una sèrie de càlculs amb unes dades d'entrada (paràmetres), per tal d'acabar retornant un resultat
- Exemple:

```
int factorial(int n) {
    int res = 1; int i;
    if(n < 0){
        printf("Error: valor n < 0!!!\n");
        return(-1);
    }else{
        for(i = n; i >= 2; i--) {
            res *= i;
        }
        return(res);
    }
}
```

Capçalera de la funció : tipus de retorn + nom de la funció + (llista de paràmetres)

Declaració de variables locals de funció

Retorn del resultat obtingut (en qualsevol cas) => sentència " return "

Definició i crida a funció (2/3)

- **Una funció sempre realitza les mateixes operacions, però les dades d'entrada** (valor específic dels paràmetres) **poden canviar**
- Mitjançant el **mecanisme de pas de paràmetres**, quan cridem a una funció podem proporcionar-li el valor que prendrà cadascun d'ells quan es comenci a executar
- Exemples:

```
int resultat = factorial(4);
printf("4! = %d\n", resultat);
/*...*/
printf("10! = %d\n", factorial(10));
/*...*/
int valor = 8;
printf("%d! = %d\n", valor, factorial(valor));
```

Cridem a la funció passant-li una constant literal com a paràmetre (n = 4, dins de la funció); guardem resultat i ho mostrem posteriorment

En aquest cas, directament mostrem el resultat retornat per la funció

Alternativament, podem passar-li una variable que encaixi amb el tipus del paràmetre definit per a la funció

Definició i crida a funció (3/3)

- **Quan cridem a una funció**, aquesta **cal que estigui definida prèviament**, de forma que el compilador en pugui conèixer les seves característiques (tipus de retorn, nom i llista de paràmetres):

```
#include<stdio.h>
#include<stdlib.h>

int factorial(int n) {
 /* Implementació de la funció (transparència anterior) */
}
int main() {
   int resultat = factorial(4);
   printf("4! = %d\n", resultat);
   return(EXIT_SUCESS);
}
```

Cridem la funció prèviament definida

Funcions void

- Per a la definició d'aquelles **funcions que no retornen cap dada en acabar l'execució**, s'utilitza el **tipus de retorn void**. Per exemple:

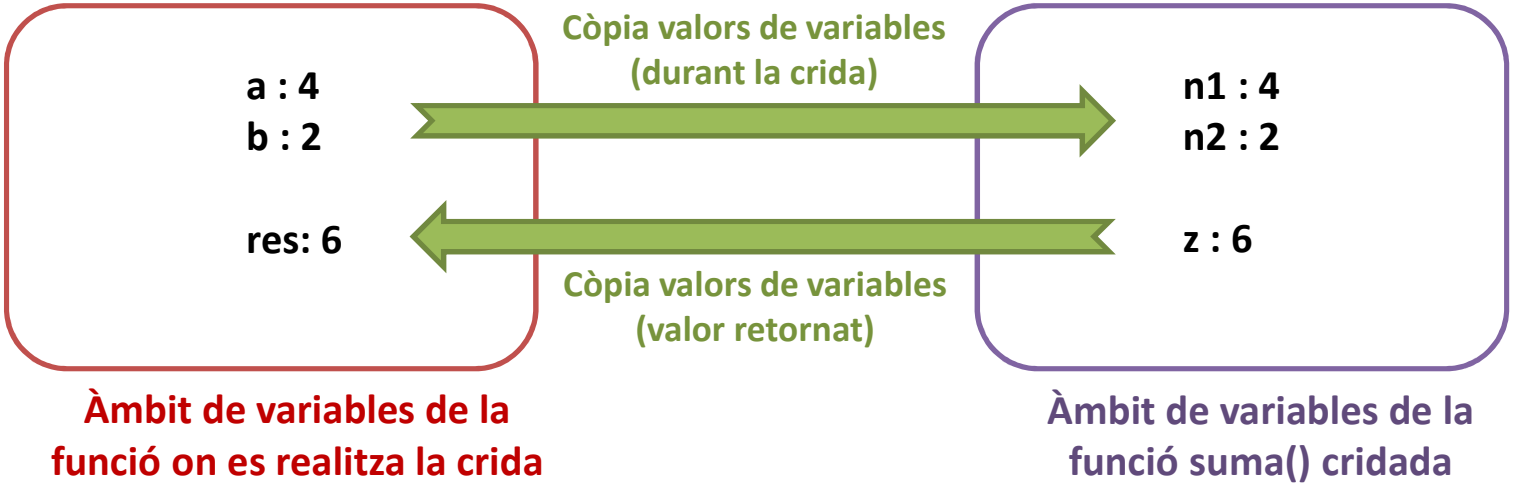
```
#include<stdio.h>
#include<stdlib.h>
#define MAX_CHAR 50
void saluda(char nom[]) {
    printf("Hola %s", nom);
}
int main(){
    char nom[MAX_CHAR];
    printf("Com et dius?\n");
    fgets(nom, MAX_CHAR, stdin); /* Guarda el '\n' */
    saluda(nom);
    return(EXIT_SUCCESS);
}
```

Pas de paràmetres per valor (1/2)

- El **pas de paràmetres a funció** en C s'anomena **per valor**, doncs **implica** una **còpia de valors** entre els àmbits de les funcions
- Per exemple, considerant el fragment de codi:

```
int a = 4, b = 2;
int res = suma(a, b);
```

```
int suma(int n1, int n2) {
    int z = n1 + n2;
    return(z);
}
```



Pas de paràmetres per valor (2/2)

- Les funcions treballen amb els valors dels seus paràmetres, que són **còpies** de valors de variables definides a les funcions des d'on es criden
  - Qualsevol **canvi sobre els paràmetres** d'una funció **NO afectarà a les variables de la funció des d'on es crida**
- Les **variables locals i els paràmetres d'una funció** s'alliberen de memòria **quan aquesta finalitza**. **Qualsevol valor** obtingut dins de la funció **que es vulgui guardar ha de ser retornat**
- **Existeix una excepció**: **els paràmetres de tipus vector o matriu**
  - En comptes de duplicar-ne tot el contingut, **es passa la seva @ de memòria com a paràmetre**. Això implicarà que la **modificació del seu contingut sí que es veurà reflectit,** un cop la funció finalitzi

Prototips de funció (1/2)

- Per assegurar-nos que el codi de la funció main() anterior funcioni, hem definit la funció factorial() abans d'aquesta
- Una opció més flexible, que ens permetrà implementar les funcions a qualsevol lloc del programa, són els **prototips de funció**
- El prototip d'una funció és **similar a la seva capçalera**, on s'especifica el seu tipus de retorn, nom i tipus de tots els paràmetres (acabant amb ";"). El seu objectiu és **informar el compilador de les característiques de la funció**
- Una organització de fitxer de codi font C senzill podria ser la següent:
  1. Línies amb directives #include i #define
  2. Definició de structs i sinònims de tipus (typedef)
  3. **Prototips (declaracions) de funcions**
  4. Implementació de la funció principal main()
  5. **Implementació de les funcions prototipades prèviament**

Prototip de funció (2/2)

- Exemple :

```
#include<stdio.h>
#include<stdlib.h>

int factorial(int);   /* Prototip de la funció factorial() */

int main() {
    int resultat = factorial(4);     /* Crida a factorial() */
   printf("4! = %d\n", resultat);
   return(EXIT_SUCESS);
}

int factorial(int n) {
   /* Implementació de la funció (transparència anterior) */
}
```

Fitxers de capçalera

- Els prototips de funció ens permeten implementar les funcions en qualsevol punt del nostre codi. Tanmateix:
  - Si s'acaben cridant des de diversos fitxers de codi font, haurem d'afegir els seus prototips a tots ells
  - Si canviem la capçalera d'una funció, haurem d'actualitzar tots els prototips prèviament afegits
- El llenguatge C introdueix els anomenats fitxers de capçalera (*header files* en anglès), amb extensió `.h`:
  - Contenen la definició de totes les estructures de dades i capçaleres (prototips) de funcions relacionades amb un àmbit de funcionalitat
  - S'implementen al fitxer amb extensió `.c` amb mateix nom
  - Qualsevol fitxer que utilitzi les estructures de dades i/o funcions només cal que inclogui el fitxer de capçalera mitjançant `#include "..."`

Exercicis - I

- **<u>EXERCICI 2.1</u>** : Implementa el fitxer de capçalera `complex.h`, que contingui la definició de l'estructura de dades `t_complex` i les capçaleres de funcions següents:

```
typedef struct{
    float real;
    float imaginaria;
}t_complex;

t_complex llegir_complex();
void mostrar_complex(t_complex c);
float calcular_mod(t_complex c);
float calcular_arg(t_complex c);
```

Implementa al fitxer de codi font `complex.c` la funció: `t_complex llegir_complex();` aquesta funció llegeix la part real i imaginària d'un nou nombre complex (en format a+bi), per tal d'inicialitzar després una nova variable de tipus `t_complex` que retornarà.

Exercicis - I

- **<u>EXERCICI 2.2</u>** : Implementa ara la funció: `void mostrar_complex (t_complex c);` aquesta funció ha de mostrar la part real i imaginària del complex `c` passat com a paràmetre. Exemples de sortida esperada:

```
2.0 +2.0i
1.0 -3.0i
3.0i
-4.0
```

- **<u>EXERCICI 2.3</u>** : Implementa ara la funció: `float calcular_mod(t_complex c);` aquesta funció ha de calcular el mòdul del complex `c` passat com a paràmetre i retornar-lo.

- **<u>EXERCICI 2.4</u>** : Implementa ara la funció: `float calcular_arg(t_complex c);` aquesta funció ha de calcular l'argument en radians (entre [-pi, pi]) del complex `c` passat com a paràmetre i retornar-lo.

Exercicis - I

- **<u>EXERCICI 2.5</u>**: Implementa la funció `int main();` del programa per tal de comprovar el correcte funcionament de les funcions implementades. A continuació es mostren un parell d'exemples d'execució esperats:

```
Introdueix un nombre complex(format: a+bi): 0.0+2.0i

El nombre complex introduit es: 2.00i
El seu modul es: 2.000000
El seu argument es: 1.570796 radians

Introdueix un nombre complex(format: a+bi): 2.0+-2.0i

El nombre complex introduit es: 2.00 -2.00i
El seu modul es: 2.828427
El seu argument es: -0.785398 radians
```

Pas de paràmetres per referència (1/2)

- Molts cops ens interessarà que la funció implementada actuï directament sobre les mateixes variables definides dins l'àmbit de la funció on ha estat cridada, no pas sobre còpies d'aquestes
- Exemple:

```
void swap(int a, int b) {
   int aux = a;
   a = b;
   b = aux;
}
/* ... */
int val1 = 5; int val2 = 4;
swap(val1, val2);
printf("val1: %d; val2: %d\n", val1, val2);
```

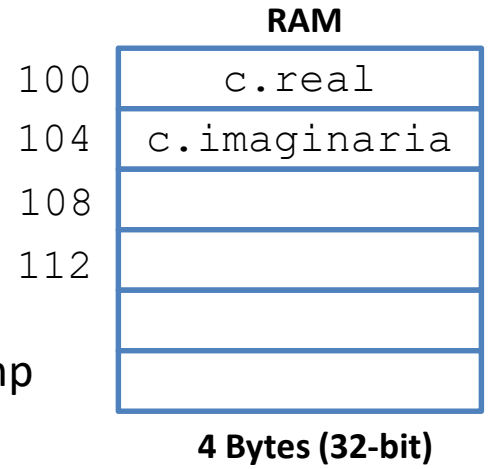
Quins valors s'acaben mostrant per pantalla??

Pas de paràmetres per referència (2/2)

- La solució consisteix en **passar paràmetres per referència**. És a dir, **passar com a paràmetre l'adreça de memòria** on està guardada **la variable de la funció que ha efectuat la crida**
- Això **es realitza en C mitjançant punters**, uns tipus de variables que guarden les @memòria on estan emmagatzemats els valors d'altres variables
  - Un **paràmetre d'una funció** es pot **declarar com a punter a un cert tipus de dada**
  - **En comptes de passar una còpia del valor d'una variable**, **la funció passarà la seva @memòria**, **que el paràmetre guardarà** en el moment d'iniciar l'execució de la funció
  - **Actuant adequadament sobre el paràmetre**, **la funció podrà modificar el valor de la variable original**

Punters (1/6)

- Durant la **declaració d'una variable, es reserva espai en memòria**, prou gran com **per enmgatzemar-hi un valor del tipus declarat**. Per exemple:
  - char : 1 Byte
  - int : 4 Bytes
  - float : 4 Bytes
  - double : 8 Bytes
  - vector: Dimensió*mida element (posició)
  - struct : Suma de les mides requerides per cada camp
- **La @memòria d'una variable que ocupa més d'1 Byte, és la més baixa de tot l'espai reservat**




```
typedef struct{
    float real;
    float imaginaria;
}t_complex;

/* ... */
t_complex c;
```

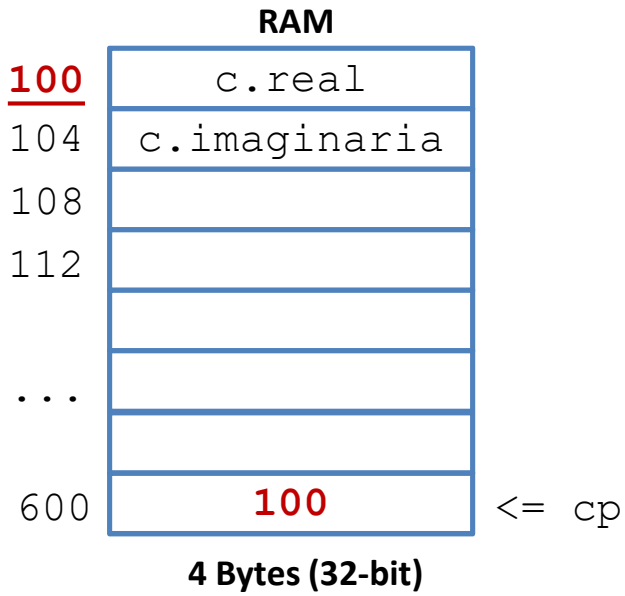
Punters (2/6)

- Per a **tot tipus de dades "T"**, **existeix un tipus de dades que anomenarem "punter a T"** i que escriurem com a **"T *"**. Exemples:
  - char : char *
  - int : int *
  - float : float *
  - double : double *
  - t_complex : t_complex *
- **L'espai en memòria requerit per emmagatzemar el valor d'una variable difereix segons el seu tipus**. Però l'espai en memòria requerit **per emmagatzemar un "punter a T" sempre serà el mateix** , p.e., 4 Bytes
- Com a bona pràctica, **convé inicialitzar tots els punters declarats a NULL** (punter nul). D'aquesta forma **podrem detectar fàcilment quan un punter apunta realment a una @memòria vàlida**

Punters (3/6)

- En llenguatge C, podem **obtenir la @memòria d'una variable** mitjançant l'**operador &** seguit del seu nom, que guardarem en una variable punter:

```
typedef struct{
   float real;
   float imaginaria;
}t_complex;

/* ... */
t_complex c;
c.real = 2.0;
c.imaginaria = 4.0;
t_complex *cp = &c;
```

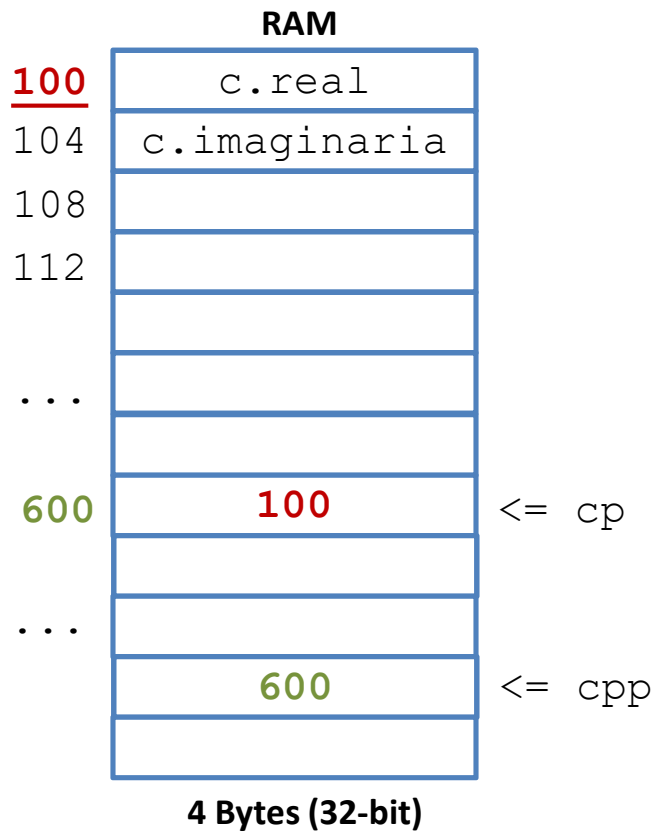


- Hem obtingut un punter a la variable `c` de tipus `t_complex` mitjançant `&c`, que l'ordinador ha decidit desar a la @memòria = 600
- **Actuant sobre el punter "cp" podrem accedir indirectament a "c"**

Punters (4/6)

- **El llenguatge C permet múltiples nivells d'indirecció**: podem definir un punter a punter a T, és a dir T ** (guarda la @memòria d'un punter a T)

```
typedef struct{
   float real;
   float imaginaria;
}t_complex;

/* ... */
t_complex c;
c.real = 2.0;
c.imaginaria = 4.0;
t_complex *cp = &c;
t_complex **cpp = &cp;
```



- Això ens permetria, per exemple, **passar un punter per referència a una funció**

Punters (5/6)

- **A més de l'operador &**, que ens permet obtenir la @memòria on està emmagatzemada una variable (fins i tot un punter), **disposem també de l'operador ***, que ens permet desfer una indirecció
- L' operador * **ens permetrà accedir al contingut de l'espai en memòria apuntat per un punter, per tal de modificar-lo**
- Per exemple:

```
int val = 50;
int *valp = NULL;
valp = &val; /* valp apunta a @memòria de val */
printf("val = %d; valp = %p \n", val, valp);
printf("val = %d; *valp = %d \n ", val, *valp);
*valp = 25; /* Canviem val desfent indirecció */
printf("val = %d; *valp = %d \n", val, *valp );
```

Punters (6/6)

- En el cas d'un **punter a estructura**, **podem seguir utilitzant l'operador * per tal d'accedir a els seus camps**. Per exemple:

```
t_complex c;
t_complex *cp = &c;
(*cp).real = 2.0;
(*cp).imaginaria = 4.0;
```

- **També podem usar l'operador ->. En comptes de (*punter).camp** , escriurem **punter->camp**. Per exemple, el codi anterior quedaria:

```
t_complex c;
t_complex *cp = &c;
cp->real = 2.0;
cp->imaginaria = 4.0 ;
```

Funció swap correcta

- Definint paràmetres de tipus punter (permetent el pas de paràmetres per referència) aconseguirem una funció swap totalment funcional:

```
void swap(int *a, int *b){

     int aux = *a;
     *a = *b;
     *b = aux;
}
 /* ... */
 int val1 = 5; int val2 = 4;
 /* Passem @memòria de val1 i val2 com a paràmetre */
 swap(&val1, &val2);
 printf("val1: %d; val2: %d\n", val1, val2);
```

Copiem contingut guardat a l'espai de memòria apuntat per "a" a "aux"

Copiem contingut de l'espai de memòria apuntat per "b" a aquell apuntat per "a"

Copiem el valor de "aux" a l'espai de memòria apuntat per "b"

Exercicis - II

- **<u>EXERCICI 2.6:</u>** Declara a `complex.h` i implementa a `complex.c` la funció `void sumar(t_complex *res, t_complex c1, t_complex c2);` aquesta funció ha de sumar els nombres complexos `c1` i `c2` i guardar-ne el complex resultant a la variable de tipus `t_complex` apuntada per `res`.
- **<u>EXERCICI 2.7:</u>** De forma similar, declara i implementa les funcions:

```
void restar(t_complex *res, t_complex c1, t_complex c2);
void multiplicar(t_complex *res, t_complex c1, t_complex c2);
void dividir(t_complex *res , t_complex c1, t_complex c2);
```

- **<u>EXERCICI 2.8:</u>** Declara i implementa també la funció `void major (t_complex ** ma, t_complex *c1, t_complex *c2);` donades dues variables de tipus `t_complex` inicialitzades i una altra de tipus punter a variable `t_complex`, aquesta funció accepta referències a les 3 com a paràmetre. Un cop executada, la funció ha d'assegurar que la variable de tipus punter apunta al major dels dos complexos, comparant-ne els mòduls.

Funcions amb vectors

- **Els vectors (i matrius) sempre es passen per referència** (es passa la seva @memòria en comptes d'una còpia del contingut sencer). **No cal declarar el paràmetre com a punter a vector**, doncs ja s'assumeix per defecte
- De fet, **un vector és**, en realitat, **un punter a la primera posició d'aquest** . Per tant, **un vector pot ser tractat com a punter quan es desitgi**:

```
int v[5]={1,2,3,4,5};  /* Reserva espai i inicialitza 5 int's */
int *vp = v; /* Amb vp apuntem v[0]. Equiv.: int *vp = &v[0]; */
int i;
for(i=0; i < 5; i++) {
    printf("%d\n, vp[i]);
}
```

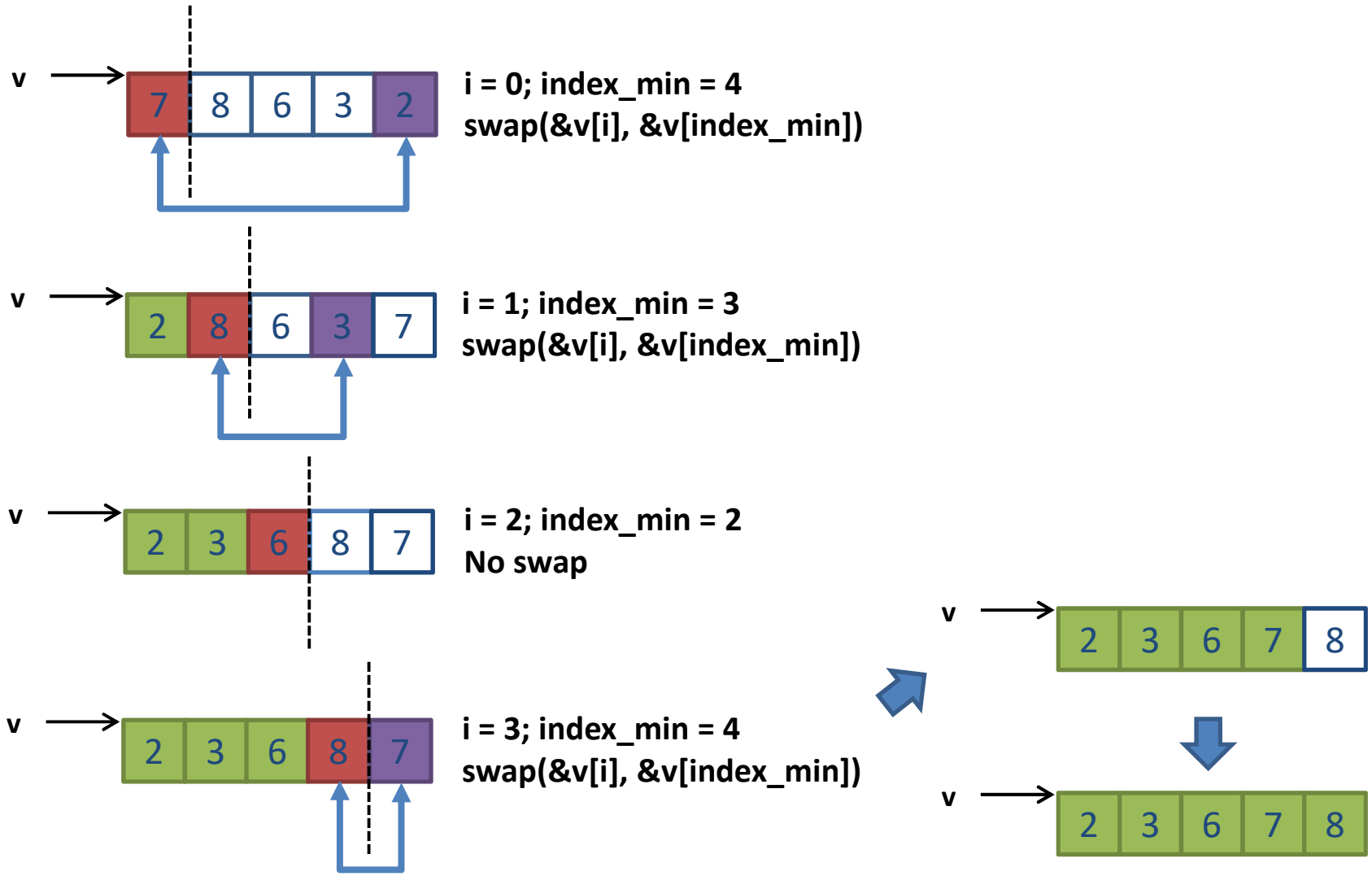
- **Una funció no pot retornar un tipus vector (o matriu)** : sempre cal passar-lo com a paràmetre i es modifica dins de la funció

Algorismes d'ordenació: Selection Sort (1/3)

- L'algorisme d'**ordenació per selecció directa** (**Selection Sort**) és molt probablement la **forma més senzilla d'ordenar un vector**
- El seu **funcionament** és el següent:
  - Busquem el menor element i l'intercanviem pel primer del vector
  - El segon menor element l'intercanviem pel segon del vector
  - El tercer menor element menor l'intercanviem pel tercer del vector
  - I així successivament, fins a tenir tot el vector ordenat de forma creixent
- **A cada iteració** de l'algorisme, no cal **comprovar** TOTS els elements del vector, tan sols **aquells no ordenats encara**
- **Buscarem el menor element des de la posició "i" fins al final del vector, intercanviant-lo pel de la posició "i" si és menor que aquest**

Exemple animat : https://es.wikipedia.org/wiki/Ordenamiento_por_selección

Algorismes d'ordenació: Selection Sort (2/3)



Algorismes d'ordenació: Selection Sort (3/3)

- El codi de l'algorisme seria:

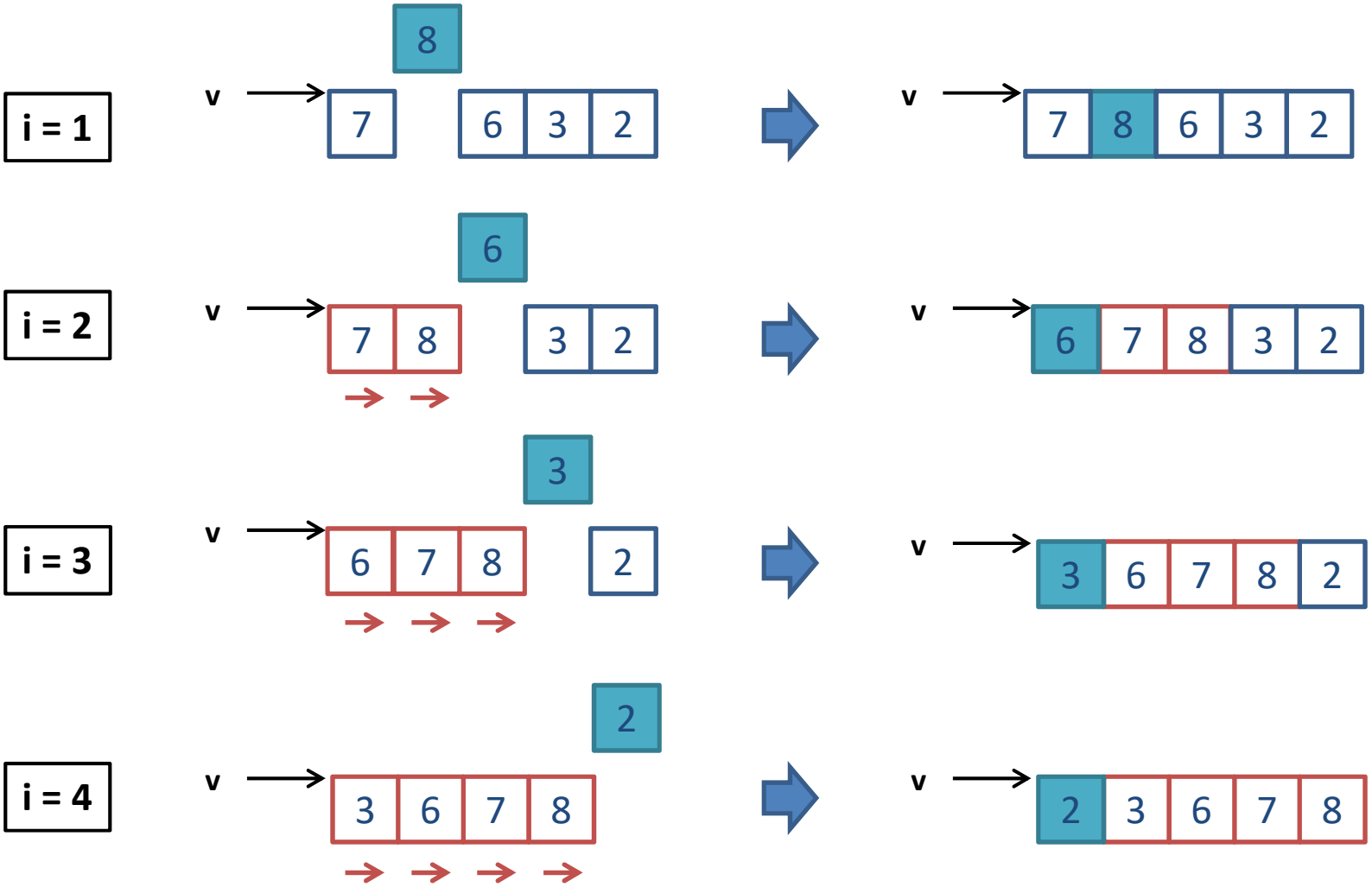
```
void swap(int *a, int *b){ ...}

void selection_sort(int v[NUM_POSICIONS])
{
     int i, j, index_min;
     for(i = 0; i < NUM_POSICIONS-1; i++) {
         index_min = i;
         for(j = i+1; j < NUM_POSICIONS; j++) {
             if (v[j] < v[index_min])
                 index_min = j; /* Actualitzem index_min */
         }
         if(index_min != i)
             swap(&v[i], &v[index_min]);
     }
}
```

Algorismes d'ordenació: Insertion Sort (1/3)

- L'algorisme d'**ordenació per inserció** (**Insertion Sort**) contempla **una altra forma d'ordenar un vector**, molt natural per a l'ésser humà:
  - L'algorisme comença per l'element a la posició 1 del vector i el guarda
  - Després, recorre els elements anteriors a l'element desat enrere, cap al principi del vector, desplaçant-los una posició endavant si aquests són més grans que l'element desat
  - En cas de trobar un element menor a l'element desat, l'algorisme copia l'element inicialment desat a la posició següent
  - Aquest procediment es realitza per a tots els elements del vector, des de l'element situat a la posició 1 fins a l'element ubicat a la darrera posició del vector

Exemple animat : https://es.wikipedia.org/wiki/Ordenamiento_por_inserción

Algorismes d'ordenació: Insertion Sort (2/3)



Algorismes d'ordenació: Insertion Sort (3/3)

- El codi de l' algorisme seria:

```
void insertion_sort(int v[NUM_POSICIONS]) {
    int i, j, valor;
    for(i = 1; i < NUM_POSICIONS; i++) {
            valor = v[i];
            j = i-1;
            while(j >= 0 && v[j] > valor) {
                    v[j+1] = v[j]; /* Desplacem element */
                    j--;
            }
            v[j+1] = valor;
    }
}
```

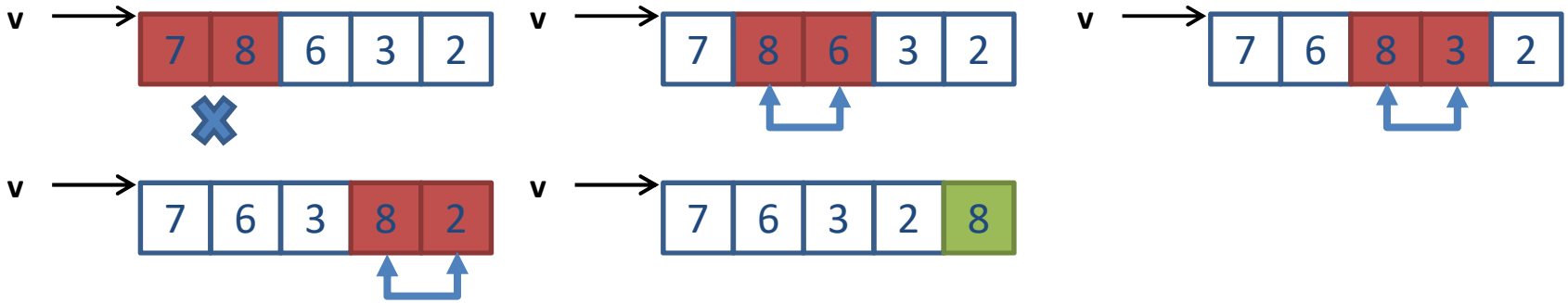
Algorismes d'ordenació: Bubble Sort (1/4)

- L'algorisme d'**ordenació de la bombolla** (**Bubble Sort**) defineix una tercera **forma d'ordenar un vector molt coneguda** :
  - L'algorisme comença des del principi del vector, comparant parells d'elements consecutius (0-1, 1-2, 2-3, ...), intercanviant-los si estan en ordre incorrecte (si primer > segon)
  - Havent comparat (i intercanviat si cal) tots els parells d'elements, haurem enviat així l'element més gran al final del vector
  - Aquesta operació s'efectua més vegades, enviant el segon element més gran a la penúltima posició del vector, el tercer a l'avantpenúltima, etc...
- L'algorisme finalitza quan no s'itercanvia cap parell d'elements consecutius al llarg d'una iteració (vector ja ordenat), o en completar el nombre màxim d'iteracions (= nombre d'elements -1)

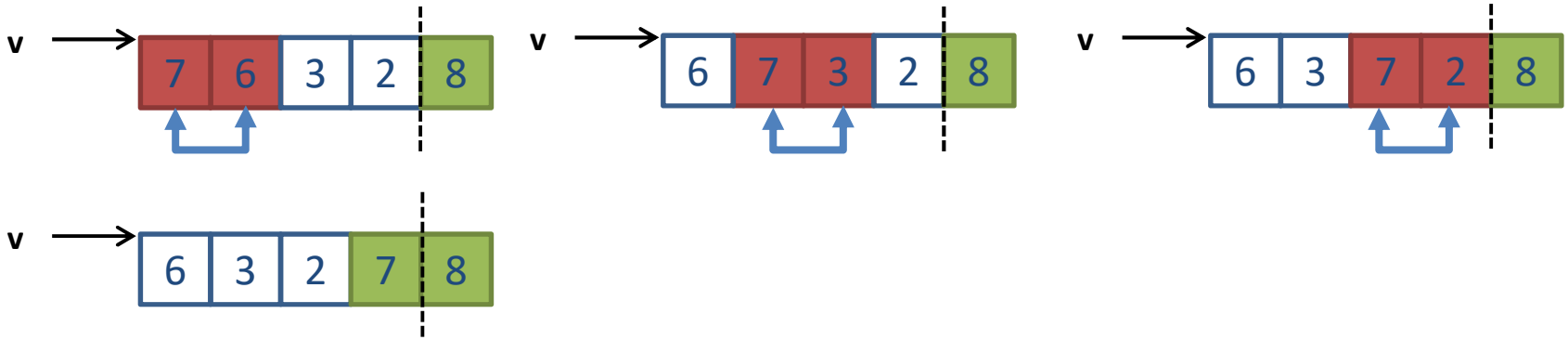
Exemple animat : https://es.wikipedia.org/wiki/Ordenamiento_de_burbuja

Algorismes d'ordenació: Bubble Sort (2/4)

i = 0

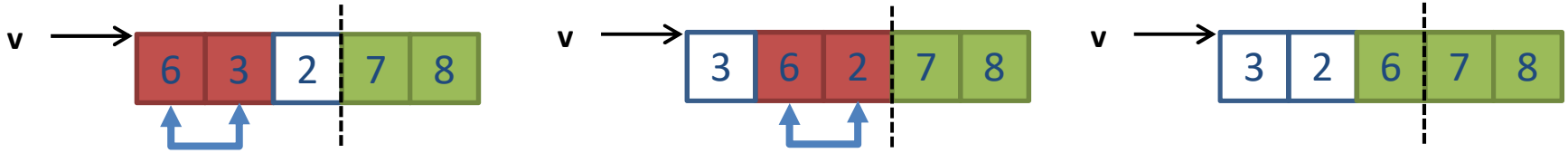


i = 1

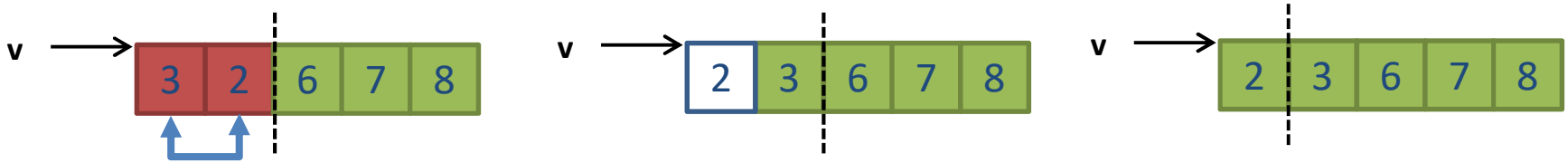


Algorismes d'ordenació: Bubble Sort (3/4)

i = 2



i = 3



Algorismes d'ordenació: Bubble Sort (4/4)

- El codi de l'algorisme seria:

```
void swap( int *a, int *b){ ...}

void bubble_sort(int v[NUM_POSICIONS]) {
    int i = 0, j, inter = 1;
    while(i < NUM_POSICIONS - 1 && inter) {
        inter = 0;
        for(j = 1; j < NUM_POSICIONS - i; j++) {
            if(v[j-1] > v[j]) {
                swap (&v[j-1], &v[j]);
                inter = 1;
            }
        }
        i++;
    }
}
```

Algorisme de cerca binària (1/3)

- Treballant amb **vectors ordenats**, existeixen **algorismes de cerca més eficients que la cerca seqüencial** bàsica. Un exemple d'aquests és l'anomenada **cerca binària** (o dicotòmica)
- Partint d'un vector ordenat de forma creixent, el funcionament és:
  - Es guarden els índexs del menor (inf) i major (sup) element del vector. Es calcula l'índex de l'element central del vector (centre) = (inf+sup)/2
  - Si l'element central és el cercat, element trobat i final. Sinó, si element buscat < que l'element central, sup = centre – 1 (hauria d'estar ubicat a la meitat inferior del vector). Sinó, inf = centre + 1 (hauria d'estar ubicat a la meitat superior del vector). Recalculem centre de nou
  - Aquesta operació es realitza mentre l'element a la posició centre != cercat && inf <= sup. Si en un determinat moment arribem a inf > sup, l'element buscat no existeix al vector

Algorisme de cerca binària (2/3)

Cerca satisfactòria: conté número 8?

inf = 0; sup = 5; centre = (0+5)/2 = 2

8 == v[centre] -> NO
8 < v[centre] -> NO
8 > v[centre] -> SÍ -> inf = 3; sup = 5;

centre = (3+5)/2 = 4

8 == v[centre] -> SÍ!

Número trobat!

Cerca fallida: conté número 7?

inf = 0; sup = 5; centre = (0+5)/2 = 2

7 == v[centre] -> NO
7 < v[centre] -> NO
7 > v[centre] -> SÍ -> inf = 3; sup = 5;

centre = (3+5)/2 = 4

7 == v[centre] -> NO
7 < v[centre] -> SÍ -> inf = 3; sup = 3

centre = (3+3)/2 = 3

7 == v[centre]
7 < v[centre] -> NO
7 > v[centre] -> SÍ -> inf = 4; sup = 3

No s'ha trobat!

Algorisme de cerca binària (3/3)

- El codi de l'algorisme seria:

```
int binary_search(int v[NUM_POSICIONS], int buscat) {
    int inf = 0, sup = NUM_POSICIONS - 1, centre;
    int found = -1;
    while(inf <= sup && found == -1) {
        centre = (inf + sup)/2;
        if (cercat == v[centre])
            found = centre;         /* Guardem índex a found */
        else if(cercat < v[centre])
            sup = centre - 1;
        else
            inf = centre + 1;
     }
     return(found);
}
```

Exercicis - III

- **<u>EXERCICI 2.9</u>** : Afegeix al fitxer de capçalera `complex.h` prèviament implementat el següent struct:

```
#define MAX_COMPLEXOS 20

typedef struct{
    int num_comp;
    t_complex complexos[MAX_COMPLEXOS];
}t_vec_comp;
```

Declara també a `complex.h` i implementa a `complex.c` la següent funció: `t_vec_comp llegir_vector_complexos();` aquesta funció sol·licita el nombre de complexos que cal llegir i desar en una nova variable de tipus `t_vec_comp` que retornarà. El nombre ha d'estar comprès entre `[1, MAX_COMPLEXOS]`. Sinó, la funció ha de notificar-ho i tornar a sol·licitar el nombre. Per a llegir cada complex utilitza la funció `t_complex llegir_complex();` implementada a l'exercici 2.1.

Exercicis - III

- **<u>EXERCICI 2.10</u>** : Implementa la funció: `void mostrar(t_vec_comp vc);` aquesta funció ha de mostrar per pantalla tots els nombres complexos continguts a `vc`. Si el vector és buit, la funció ha de mostrar un missatge indicant-ho. Per mostrar cada complex, pots emprar la funció `void mostrar_complex(t_complex c);` codificada a l'exercici 2.2.
- **<u>EXERCICI 2.11</u>** : : Implementa la funció: `void ordenar_seleccio (t_vec_comp *vc);` aquesta funció ha d'ordenar mitjançant l'algorisme d'ordenació per selecció directa els nombres complexos desats a `*vc` de menor a major comparant-ne els mòduls.
- **<u>EXERCICI 2.12</u>** : De forma similar, implementa les següents funcions que ordenen mitjançant els algorismes d'ordenació per inserció i de la bombolla els nombres complexos guardats a `*vc` de menor a major comparant els seus mòduls : `void ordenar_insercio(t_vec_comp *vc);` i `void ordenar_bombolla(t_vec_comp *vc);`.

Exercicis - III

- **<u>EXERCICI 2.13</u>** : Implementa la funció: `int cerca_binaria(t_vec_comp vc, t_complex c);` aquesta funció assumeix que el vector `vc` està ordenat de forma creixent en funció del mòdul de cada nombre complex. Així, realitza una cerca binària per trobar si al vector existeix un complex amb mateix mòdul al de `c`, retornant-ne la seva posició; sinó retorna -1.
- **<u>EXERCICI 2.14</u>** : Implementa la funció: `int inserir_ordre(t_vec_comp *vc, t_complex c);` aquesta funció insereix el nou complex `c` a `*vc`, de manera que es mantingui l'ordre creixent en aquest. La funció retorna 0 si el complex s'ha pogut inserir o -1 si el vector no disposa de capacitat suficient per allotjar-lo.
- **<u>EXERCICI 2.15</u>** : Implementa la funció: `int eliminar(t_vec_comp *vc, t_complex c);` aquesta funció elimina el complex `c` del vector ordenat de complexos `*vc`, si aquest s'hi troba. En cas afirmatiu, la funció retorna 0. En cas contrari, retorna -1.